

5. Le moteur d'exécution (transaction ACID)

Étape 2 — EILMS : Database Persistence Layer (JDBC, PostgreSQL, JPA/Hibernate)

Fichier concerné : src/main/java/com/eilms/PersistenceTestMain.java

Classe de démarrage qui exécute une transaction conforme ACID via EntityManager, pour vérifier que l'ensemble de la chaîne (Java → Hibernate → PostgreSQL) fonctionne réellement.

```
public class PersistenceTestMain {
    public static void main(String[] args) {

        // 1. Construire l'EntityManagerFactory pour l'unité de persistance définie
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("eilms-pu");

        // 2. Obtenir une instance active d'EntityManager
        EntityManager em = emf.createEntityManager();

        // 3. Démarrer la transaction
        em.getTransaction().begin();

        // 4. Instancier et persister l'entité de domaine
        WarehouseItemEntity item = new WarehouseItemEntity("ITEM-201", "PostgreSQL Server Node", 25);
        em.persist(item);

        // 5. Valider la transaction (écriture réelle dans PostgreSQL)
        em.getTransaction().commit();

        System.out.println(">>> Entity successfully persisted to PostgreSQL database!");

        // 6. Nettoyage des ressources
        em.close();
        emf.close();
    }
}
```

Explication détaillée

EntityManagerFactory : objet « lourd » et thread-safe, créé une seule fois par application. Il lit persistence.xml, construit le modèle de métadonnées, et sert d'usine à EntityManager.

EntityManager : objet « léger », NON thread-safe (il faut en créer un par thread/transaction). Il gère le contexte de persistance : quels objets sont « managés », « détachés », « supprimés », etc.

em.getTransaction().begin() / commit() : délimitent explicitement la transaction. Toute modification (persist, merge, remove) doit se faire entre begin() et commit() pour garantir les propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité). Si une erreur survient avant commit(), on peut faire rollback() pour tout annuler.

em.persist(item) : marque l'objet comme « managé » — Hibernate va générer et exécuter l'INSERT SQL correspondant au moment du commit.

em.close() / emf.close() : libèrent les ressources (connexions, cache de session). Toujours fermer l'EntityManager après usage, et l'EntityManagerFactory à la fin de l'application.

Analyse pédagogique du professeur

EntityManagerFactory : objet usine lourd et thread-safe qui analyse persistence.xml et construit les modèles de métadonnées. Créé une seule fois par cycle de vie de l'application.

Contexte EntityManager : contexte de persistance léger, non thread-safe, qui gère l'état des entités (Transient, Managed, Detached, Removed).

Limites de transaction explicites : les modifications relationnelles doivent avoir lieu à l'intérieur de frontières explicites begin() et commit() pour garantir les propriétés ACID de la base de données.

Vérification à l'exécution

Exécuter PersistenceTestMain.java produit du SQL formaté dans la console et crée la structure relationnelle correspondante dans PostgreSQL :

```

Hibernate:
    create table warehouse_items (
        stock_quantity integer not null,
        item_id varchar(50) not null,
        item_name varchar(150) not null,
        primary key (item_id)
    )

Hibernate:
    insert
    into
        warehouse_items
        (item_name, stock_quantity, item_id)
    values
        (?, ?, ?)

>>> Entity successfully persisted to PostgreSQL database!
```

item_id (PK)	item_name	stock_quantity
ITEM-201	PostgreSQL Server Node	25